VERIQTA

VERIQTA SUBSCRIBER SERIES

CASE FILE #003

THE DATABASE THAT TOOK DOWN PRODUCTION

How One Query, One Bottleneck, and One Assumption Caused Hours of Downtime

USERS HEALTHY

KUBERNETES HEALTHY

CPU NORMAL

MEMORY NORMAL

EVERYTHING LOOKED HEALTHY.
THE DATABASE WASN'T.

NETWORK HEALTHY

API PODS HEALTHY

DATABASE OVERLOADED

INCIDENT SNAPSHOT

STARTED	11:43 AM
USERS AFFECTED	18,742
FAILED TRANSACTIONS	2,847
RESPONSE TIME	180ms → 8.4s
SEVERITY	HIGH
REVENUE IMPACT	SIGNIFICANT

ROOT CAUSE CHAIN

MISSING INDEXES
↓
N+1 QUERIES
↓
CONNECTION EXHAUSTION
↓
MISSING CACHE
↓
NO MONITORING
↓
PRODUCTION OUTAGE

WHAT YOU'LL LEARN

- Missing Database Indexes
- N+1 Query Problem
- No Connection Pooling
- Redis Cache Layer
- No Database Monitoring
- Investigation Timeline
- New Database Architecture
- Database Readiness Checklist
- 10 Database Lessons

PRODUCTION ENGINEERING CASE STUDY

"DATABASES RARELY FAIL SUDDENLY.
THEY FAIL SLOWLY, THEN ALL AT ONCE."

THE DATABASE THAT TOOK DOWN PRODUCTION

How One Query, One Bottleneck, and One Assumption Caused Hours of Downtime

The API Was Fast.

The Database Was Drowning.

Monday
11:43 AM

Support tickets started arriving.
Customers reported:

- Slow checkout
- Failed orders
- Random timeouts
- Delayed notifications

The strange part?

- ☑ Kubernetes looked healthy
- ☑ Pods were running
- ☑ CPU usage looked normal
- ☑ Memory usage looked normal
- ☑ Load balancer was healthy

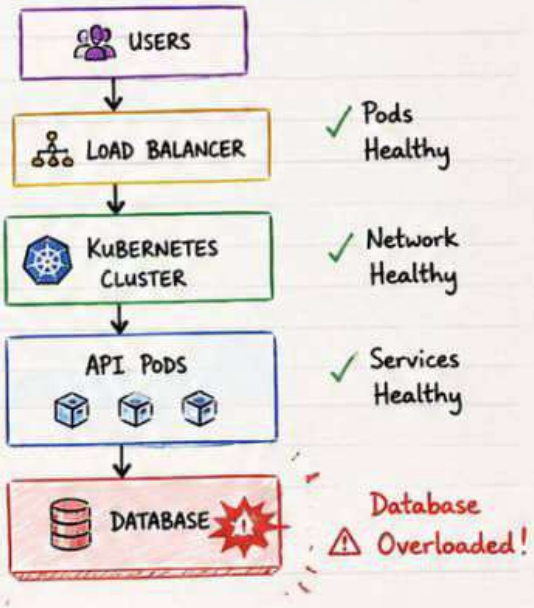
Yet users could barely use the application.

Something deeper was failing.

THE IMPACT

- Users affected: 18,742
- Average response time: 180ms → 8.4 seconds
- Failed transactions: 2,847
- Support tickets: 326
- Revenue impact: High

ARCHITECTURE (BEFORE)



INVESTIGATION JOURNEY

WHAT MADE THIS INCIDENT DIFFICULT?

- → The symptoms appeared in the application.
- → The real problem lived in the database.
- → Production outages rarely start where users feel them.

INTERVIEW NOTE

Q: Why are database problems difficult to diagnose?

A: Because application symptoms often appear before database bottlenecks become obvious.

LESSON #1

When everything looks healthy except the user experience, start looking at the DATABASE.

The bottleneck is often hidden beneath the application layer.

SERIES PROGRESS

1 Incident Overview — 2 Missing DB Indexes — 3 N+1 Query Problem — 4 No Connection Pooling — 5 Missing Cache Layer — 6 No DB Monitoring — 7 Investigation Timeline — 8 New DB Architecture — 9 DB Readiness Checklist — 10 10 DB Lessons

"Outages are never random. They are the result of small problems we didn't see coming together."

MISTAKE #1: MISSING DATABASE INDEXES

The database had to scan millions of rows because we didn't give it the right indexes.

THE PROBLEM

A critical query was running on a column with no index.

Every request triggered a FULL TABLE SCAN.

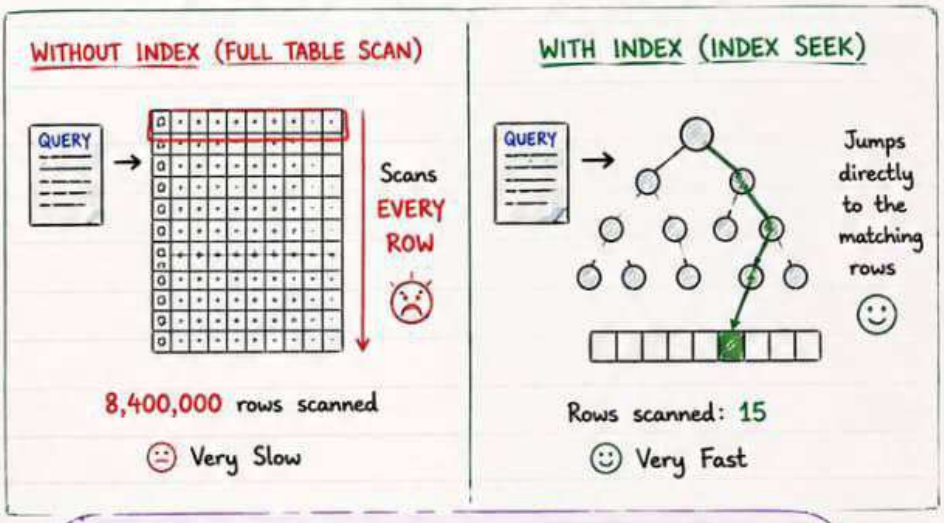
BAD QUERY (NO INDEX)

```
SELECT * FROM orders
WHERE customer_id = 12345
  AND status = 'COMPLETED';
```

WHAT HAPPENED?

- Database scanned 8.4 million rows
- CPU usage spiked
- Query took 4.2 seconds
- Connection pool exhausted
- API response time exploded

HOW THE DATABASE PROCESSED THE QUERY



Indexes reduce the amount of data the database must read. Less work for the database = Faster queries.

KEY LESSON

Databases search fast when you tell them where to look.

Indexes are like signposts for your data.

THE FIX

We created the right index based on how the query filters and sorts data.

```
CREATE INDEX idx_orders_customer_status
ON orders (customer_id, status);
```

☑ Query time dropped from 4.2s to 40ms

EXPLAIN ANALYZE (BEFORE vs AFTER)

BEFORE (FULL TABLE SCAN)

```
→ Seq Scan on orders  (cost=0.00..152830.00 rows=8400000)
  Filter: (customer_id = 12345 AND status = 'COMPLETED')
  Rows Removed by Filter: 8399985
  Execution Time: 4210.35 ms
```

AFTER (USING INDEX)

```
→ Index Scan using idx_orders_customer_status on orders
  (cost=0.43..38.75 rows=15)
  Index Cond: ((customer_id = 12345) AND (status = 'COMPLETED'))
  Execution Time: 40.21 ms
```

INTERVIEW NOTE

Q Why are indexes important in databases?

A Indexes improve read performance by allowing the database to find rows without scanning the entire table.

COMMON INDEX TYPES

- ★ PRIMARY KEY INDEX → Unique, one per table
- ★ UNIQUE INDEX → Enforces uniqueness
- ★ INDEX → Improves search performance
- ★ COMPOSITE INDEX → On multiple columns
- ★ PARTIAL INDEX → Index on a subset
- ★ COVERING INDEX → Covers the query columns

CHECKLIST

- ☑ Identify slow queries
- ☑ Check if indexes exist
- ☑ Create index on filter columns
- ☑ Verify with EXPLAIN ANALYZE
- ☑ Monitor query performance

LESSON #2

A small index today prevents a major outage tomorrow.
Design for performance, not just for functionality.

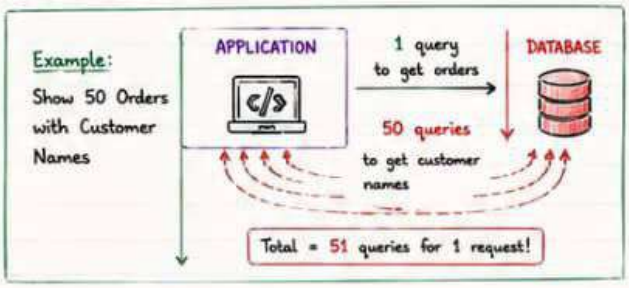
MISTAKE #2: THE N+1 QUERY PROBLEM

One request.
Hundreds of queries.
Unnecessary load on the database.

The application looked optimized, but behind the scenes, it was making way too many database calls.
Each small call seemed harmless.
Together, they brought production to its knees.

WHAT IS N+1 QUERY PROBLEM?

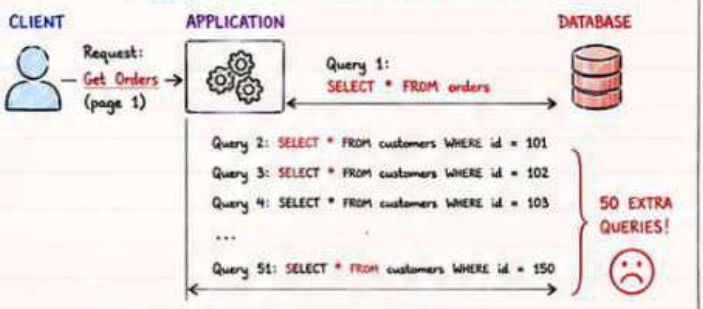
When retrieving a list of items and for each item, the application makes an additional query to fetch related data.



REAL WORLD IMPACT

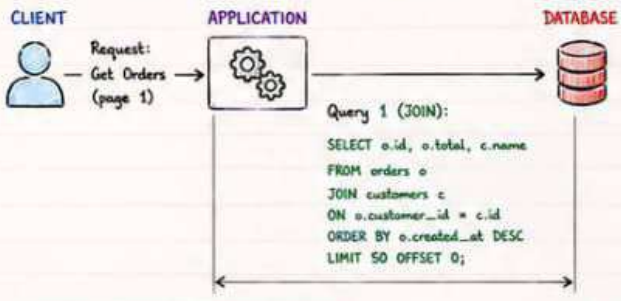
- A single page load triggered 300+ queries
- Database CPU spiked
- Connections exhausted
- Response time increased from 180ms to 8.7 seconds
- Users saw timeouts and failed checkouts

WITHOUT OPTIMIZATION (N+1 QUERIES)



Total Queries: 51
Result: Slow, costly, and painful.

WITH OPTIMIZATION (1 QUERY)



Total Queries: 1
Result: Fast, efficient, scalable.

HOW TO SPOT N+1 QUERY PROBLEM

1. Check database query logs
2. Look for repeating patterns
3. Count queries per request
4. Monitor slow query count
5. Use APM tools (New Relic, Datadog, AppSignal, etc.)

EXAMPLE SCENARIO

We showed 50 orders on a page.
For each order, we fetched the customer details separately.

→ Orders: 1 query
→ Customers: 50 queries

Total database hit: 51 queries for a single page load!

HOW TO FIX

- [x] Use JOINs to fetch related data in a single query
- [x] Use eager loading (ORM feature)
- [x] Batch queries where possible
- [x] Cache frequently accessed data
- [x] Always test with real data volume

INTERVIEW NOTE

Q: What is N+1 Query Problem?
A: It happens when the application makes 1 query to get a list of items and N additional queries to get related data for each item.

KEY LESSON

Every extra query has a cost.
Eliminate unnecessary queries.
Your future self (and your users) will thank you.

LESSON #3

Optimize data access patterns.
Small inefficiencies multiply at scale.

SERIES PROGRESS

MISTAKE #3: NO CONNECTION POOLING

Too Many Connections.
Not Enough Database.

The application created a new database connection for every single request.

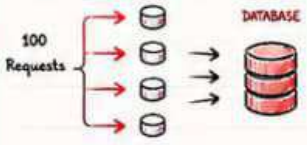
During peak traffic, the database ran out of connections.

The system started failing.

WHAT IS CONNECTION POOLING?

Instead of opening a new connection every time, we reuse existing connections from a pool.

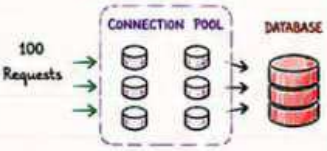
WITHOUT POOLING



100 New Connections

☹ Overwhelmed

WITH POOLING



Reuse 10 Connections

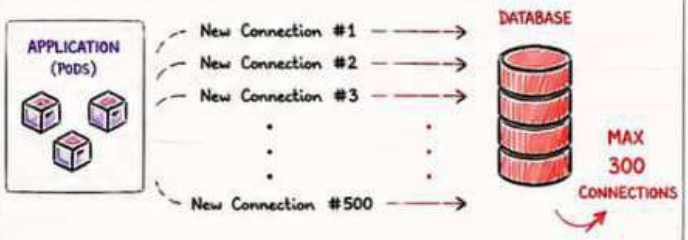
☺ Healthy & Stable

REAL WORLD IMPACT

- Connections spiked to 500+
- Database max connections: 300
- New connections were rejected
- Requests started timing out
- Cascading failures across the application
- Users saw errors and failed checkouts

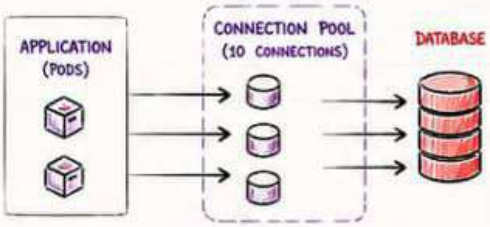
☹

WHAT HAPPENED IN PRODUCTION



After 300 connections, the database started rejecting new ones. Errors exploded. Timeouts increased. Users were impacted.

HOW CONNECTION POOLING FIXES IT



- ✓ Reuses existing connections
- ✓ Reduces overhead
- ✓ Prevents exhaustion
- ✓ Improves stability

Only a limited number of connections are created and reused. The database stays healthy under load.

TYPICAL PROBLEM SIGNS

- Too many connections
- Connection timeout errors
- "Too many clients" error
- Database CPU normal but requests failing
- Random timeouts

CHECK YOUR DATABASE METRICS

- Active Connections
- Max Connections
- Connection Rate
- Connection Errors
- Threads / Processes

EXAMPLE METRICS (BEFORE FIX)

Max Connections:	300
Active Connections:	482
Connection Errors:	126
Timeout Errors:	87
CPU: 35%	42%
DB Response Time:	8.7s

☹

EXAMPLE METRICS (AFTER FIX)

Max Connections:	300
Active Connections:	48
Connection Errors:	0
Timeout Errors:	2
CPU: 42%	42%
DB Response Time:	180ms

☺

HOW TO FIX

- ☑ Use connection pooling (HikariCP, PgBouncer, etc.)
- ☑ Set max pool size based on database capacity
- ☑ Reuse connections instead of opening new ones
- ☑ Monitor connection usage
- ☑ Test under real production load

INTERVIEW NOTE

Q: Why is connection pooling important?

A: Because databases have a limit on simultaneous connections. Pooling prevents exhaustion and improves performance.

KEY LESSON

Connections are expensive. Reuse them.

Don't create new ones unless you have to.

Efficient connections = Stable systems.

LESSON #4

Control your resources. Uncontrolled connections will take down an otherwise healthy system.

SERIES PROGRESS

1 Incident Overview · 2 Missing DB Indexes · 3 N+1 Query Problem · 4 No Connection Pooling · 5 Missing Cache Layer · 6 No DB Monitoring · 7 Investigation Timeline · 8 New DB Architecture · 9 DB Readiness Checklist · 10 10 DB Lessons

"A single resource left unchecked can bring the entire system to its knees."

MISTAKE #4: CACHE WAS MISSING

We asked the database the same question 10,000 times.

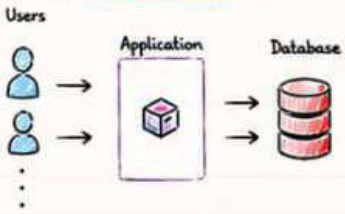
The application kept asking the database for the same data over and over.

The database did all the work. Every. Single. Time.

THE PROBLEM

Frequently accessed data was not cached. Every request, even for the same information, hit the database.

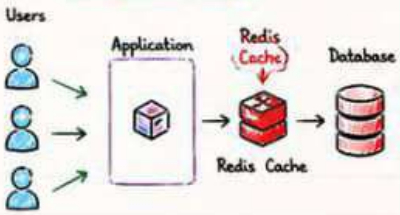
WITHOUT CACHE



Every request hits the database

☹ Slow, expensive, and wasteful

WITH CACHE (REDIS)



Most requests served from cache

☺ Fast, efficient, and scalable

EXAMPLE SCENARIO

A product page shows:

- Product details
- Price
- Inventory count
- Reviews (top 5)

These values change infrequently but are requested constantly.

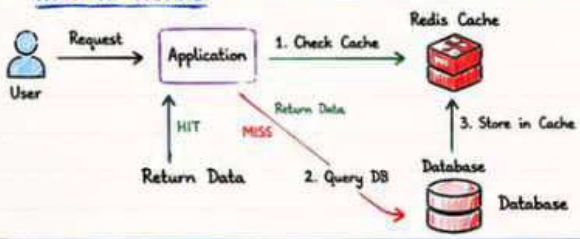
TRAFFIC BREAKDOWN (1 MINUTE)

WITHOUT CACHE	WITH CACHE
10,000 requests	10,000 requests
↓	↓
10,000 database queries	9,650 cache hits
↓	350 database queries
High Load ☹	↓
	Low Load ☺

REAL WORLD IMPACT

- Database CPU dropped from 85% to 25%
- Response time improved from 8.4s to 180ms
- Database connections reduced by 92%
- System became stable again ☺

HOW IT WORKS



HIT = Data found in cache
MISS = Data not in cache

- ✓ Faster responses
- ✓ Lower database load
- ✓ Better user experience

CACHE STRATEGIES

- ★ Cache-Aside (Lazy Loading)
- ★ Write-Through
- ★ Write-Back
- ★ TTL (Time To Live)
- ★ Invalidation on Update

WHAT WE CACHED

- ☑ Product details
- ☑ User session data
- ☑ Inventory count
- ☑ Top reviews
- ☑ Configuration data

CHALLENGES WE FACED

- Deciding what to cache
- Cache invalidation on updates
- Choosing correct TTL
- Preventing cache-staleness
- Memory usage of cache

TOOLS WE USED

- Redis (Primary Cache)
- Redis Cluster (HA)
- Key expiration (TTL)
- Eviction Policy: allkeys-lru
- Monitoring with Redis Insights

HOW WE FIXED IT

- ☑ Implemented Redis cache
- ☑ Added TTL for stale data
- ☑ Invalidated cache on updates
- ☑ Monitored hit ratio
- ☑ Tested under real load

INTERVIEW NOTE

Q: Why is caching important?

A: Caching reduces database load, improves response time, and increases system scalability.

KEY LESSON

Cache the right data.
Cache it smartly.
Invalidate it correctly.
Your database will thank you.

LESSON #5

Cache is not just a performance feature.
It is a reliability feature.
A cache can be the difference between slow... and down.

SERIES PROGRESS

1 Incident Overview — 2 Missing DB Indexes — 3 N+1 Query Problem — 4 No Connection Pooling — **5 Missing Cache Layer** — 6 No DB Monitoring — 7 Investigation Timeline — 8 New DB Architecture — 9 DB Readiness Checklist — 10 10 DB Lessons

"The best database query is the one you don't have to make."

MISTAKE #5: NO DATABASE MONITORING

The database was screaming.
Nobody heard it.

Monitoring wasn't setup for the database.

There were no alerts.

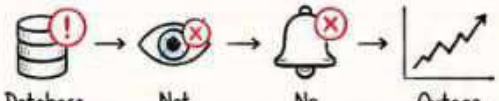
No dashboards.

No visibility.

The problem grew in the dark.

THE PROBLEM

The database showed multiple warning signs before the outage.
But we had no visibility into any of it.



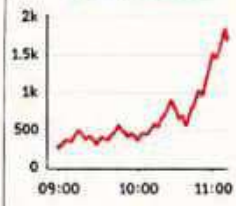
We discovered problems only after users did.

REAL WORLD IMPACT

- Slow queries increased for 2 hours before anyone noticed.
- Connections exhausted suddenly.
- Locks caused request pile-ups.
- By the time we reacted, users were already impacted.
- MTTR increased because we were blind.

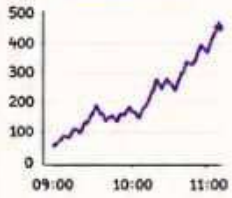
THE WARNING SIGNS WE MISSED

SLOW QUERIES



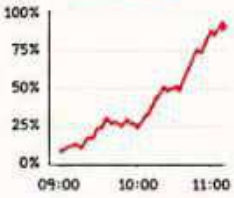
Slow queries increased 10x before outage.

CONNECTIONS



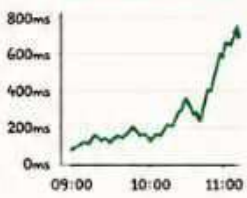
Connections reached the max limit.

CPU USAGE



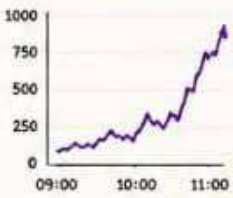
CPU usage was high but had no alert.

LOCK WAIT TIME



Lock wait time spiked causing request delays.

DISK I/O



Disk I/O was saturated by heavy queries.

All of these signals were available. We just weren't looking.

WHAT GOOD DATABASE MONITORING INCLUDES

- [x] Slow Query Monitoring
- [x] Connection Monitoring
- [x] CPU, Memory, Disk Monitoring
- [x] Lock & Wait Event Monitoring
- [x] Replication & Lag Monitoring
- [x] Query Performance Monitoring
- [x] Alerts & Anomaly Detection
- [x] Dashboards & Visibility

WHAT WE SHOULD HAVE HAD

DB Overview	Connections 312 / 300
Slow Queries 1,842 ↗	CPU Usage 82% ↗
Lock Wait Time 642 ms ↗	Replication Lag 4.2 sec ↗

A simple dashboard could have saved hours of downtime.

THE COST OF NO MONITORING

- ✗ Problems grow silently
- ✗ Longer detection time
- ✗ Higher impact
- ✗ More users affected
- ✗ Longer recovery time
- ✗ Loss of trust
- ✗ Loss of revenue

HOW TO FIX

- [x] Enable slow query log
- [x] Monitor key metrics
- [x] Create meaningful alerts
- [x] Build DB dashboards
- [x] Test alerts regularly

TOOLS WE USED

- Prometheus
- Grafana
- PMM (Percona)
- Datadog
- CloudWatch

INTERVIEW NOTE

Q: Why is monitoring critical for databases?

A: Because databases don't fail suddenly. They give signals long before they break.

KEY LESSON

You can't fix what you can't see.

Monitoring turns guesswork into visibility.

Visibility prevents outages.

LESSON #6

Monitoring is not optional. It is the earliest warning system for production outages.

SERIES PROGRESS

1 Incident Overview · 2 Missing DB Indexes · 3 N+1 Query Problem · 4 No Connection Pooling · 5 Missing Cache Layer · 6 No Monitoring · 7 Investigation Timeline · 8 New DB Architecture · 9 DB Readiness Checklist · 10 10 DB Lessons

> "Monitoring is the difference between a 5 minute fix and a 5 hour outage."

INVESTIGATION TIMELINE

Connecting The Clues

We followed the data. The data showed the truth.
The database was the real culprit.

THE INVESTIGATION JOURNEY

1. **User Reports**
   Slow checkout, timeouts, failed orders
2. **Infrastructure Checks**
   Kubernetes, pods, network, load balancer – all healthy
3. **Application Metrics**
   High latency at DB calls, normal latency elsewhere
4. **Database Deep Dive**
   Slow queries, locks, high connections, full scans
5. **Root Cause Identified**
   5 database issues working together caused the outage
6. **Fixed & Verified**
   Implemented fixes, monitored improvements, users happy

"We didn't guess.
We measured.
We followed the evidence.
That's how we found
the real problem.

TIMELINE OF INVESTIGATION

11:43 AM — Incident Starts
Support tickets spike.
Users report slow checkout and failed orders.

11:48 AM — Initial Checks
Checked Kubernetes, pods, CPU, memory, network.
Everything looked healthy.

11:52 AM — Application Metrics
APIs responding, but DB call latency is high.
Suspicion shifts to database.

11:56 AM — Database Overview
Connections near max.
CPU high. Response time climbing.

12:02 PM — Slow Query Logs
Found multiple slow queries.
Full table scans detected.
Missing indexes identified.

12:10 PM — Deeper Analysis
N+1 queries observed.
Too many connections.
Cache missing.

12:25 PM — Root Cause Confirmed
5 issues combined to overwhelm the database.
This is the real cause.

12:40 PM — Fixes Applied
Indexes added, queries optimized, pooling enabled, cache added, monitoring enabled.
System stabilizing.

01:30 PM — Users Happy Again
Performance recovered.
Tickets dropping.

WHAT THE DATA REVEALED

- ★ Slow queries were increasing
- ★ Full table scans everywhere
- ★ N+1 queries causing massive load
- ★ Connections were exhausted
- ★ No cache, same data repeated
- ★ No alerts, no visibility

KEY DATABASE METRICS DURING INCIDENT

Max Connections	482 / 500	↗
Active Connections	478	
Slow Queries (>1s)	1,842	↗
CPU Usage	85 %	↗
Average Query Time	180ms → 8.4s	↗
Lock Wait Time	642ms	↗

SLOW QUERY EXAMPLE WE FOUND

```
SELECT * FROM orders
WHERE customer_id = 12345
AND status = 'COMPLETED'
ORDER BY created_at DESC;
```

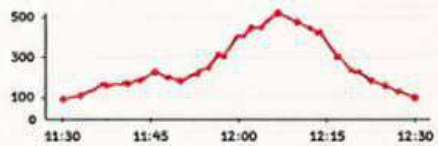
Execution Time: 4210.35 ms
Rows Scanned: 8,400,000

N+1 QUERY EXAMPLE

Page Load → 1 Request
But executed → 351 DB Queries

APPLICATION → = 351 QUERIES

CONNECTION SPIKE



TOOLS WE USED

- Prometheus
- Grafana
- Slow Query Logs
- pg_stat_statements
- Datadog
- EXPLAIN ANALYZE

EVIDENCE WE COLLECTED

- ☑ Database metrics
- ☑ Slow query logs
- ☑ Connection statistics
- ☑ Lock wait analysis
- ☑ Query execution plans
- ☑ Application traces

WHAT WE LEARNED DURING INVESTIGATION

- ★ Always follow the data
- ★ Never trust assumptions
- ★ Symptoms lie, metrics don't
- ★ The database was the bottleneck
- ★ Observability is not optional

INTERVIEW NOTE

Q: How do you investigate a database performance issue?

A: Follow the metrics, analyze queries, check connections, and find the bottleneck.

KEY LESSON

Good investigations don't start with solutions.

They start with curiosity and data.

SERIES PROGRESS

1 Incident Overview — 2 Missing DB Indexes — 3 N+1 Query Problem — 4 No Connection Pooling — 5 Missing Cache Layer — 6 No Monitoring — **7 Investigation Timeline** — 8 New DB Architecture — 9 DB Readiness Checklist — 10 10 DB Lessons

"In production incidents, the fastest way to the truth is through the data."

NEW DATABASE ARCHITECTURE

Built for Performance. Designed for Scale. Monitored for Reliability.

We didn't just fix the problems.
We rebuilt the foundation.

A strong database architecture prevents outages before they happen.

DESIGN PRINCIPLES

- ☑ Fast for the user
- ☑ Efficient for the database
- ☑ Scalable for the future
- ☑ Observable at every layer
- ☑ Resilient by design
- ☑ Cost effective

KEY IMPROVEMENTS

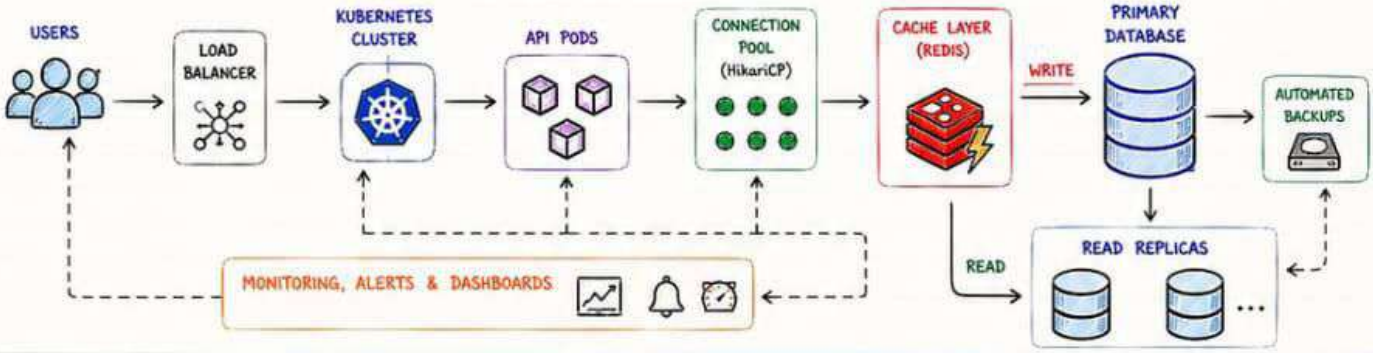
- ✓ Proper indexing strategy
- ✓ Connection pooling enabled
- ✓ Caching layer added (Redis)
- ✓ Read replicas for scale
- ✓ Query optimization
- ✓ Continuous monitoring
- ✓ Automated alerts & dashboards

ARCHITECTURE OVERVIEW

A modern, scalable, and observable database architecture.

- ★ High Performance
- ★ High Availability
- ★ High Scalability
- ★ High Visibility
- ★ High Reliability

THE NEW DATABASE ARCHITECTURE



COMPONENTS EXPLAINED

Connection Pool	→	Reuses DB connections, prevents exhaustion, improves performance.
Cache Layer (Redis)	→	Stores frequently accessed data, reduces database load.
Read Replicas	→	Handles read traffic, improves scalability and availability.
Indexes	→	Optimized indexes ensure fast and efficient queries.
Monitoring & Alerts	→	Tracks DB metrics, detects issues early, and triggers alerts.
Backups	→	Automated backups ensure data safety and quick recovery.

BEFORE vs AFTER

BEFORE	AFTER
✗ Full table scans	✓ Indexed queries
✗ N+1 queries	✓ Optimized code
✗ No connection pooling	✓ Connection pooling
✗ No cache	✓ Redis cache
✗ No monitoring	✓ Full monitoring
✗ Single DB instance	✓ Primary + Replicas
✗ No visibility	✓ End-to-end visibility
✗ Outages & slowdowns	✓ High performance

BUSINESS IMPACT

- Response time: 8.4s → 180ms
- Database CPU: 85% → 25%
- Failed transactions: 2,847 → 12
- Support tickets: 326 → 28
- Revenue impact: HIGH RISK → STABLE

HOW WE BUILT IT

- ☑ Analyzed slow queries
- ☑ Designed indexing strategy
- ☑ Implemented connection pooling
- ☑ Added Redis cache
- ☑ Setup read replicas
- ☑ Enabled monitoring & alerts
- ☑ Load tested and validated

TOOLS & TECHNOLOGIES

- PostgreSQL (Primary DB)
- Redis (Caching)
- HikariCP (Connection Pool)
- Prometheus + Grafana (Monitoring)
- pg_stat_statements (Query insights)
- Kubernetes (Orchestration)
- Terraform (IaC)

DESIGN BENEFITS

- ✓ Handles more traffic
- ✓ Scales horizontally
- ✓ Self-healing architecture
- ✓ Better user experience
- ✓ Lower cost per transaction
- ✓ Future ready

INTERVIEW NOTE

Q: What makes a database architecture production ready?

A: Performance, scalability, reliability, observability, and automation.

KEY LESSON

A good database architecture is not about using more servers.

It's about making every query count.

SERIES PROGRESS

1 Incident Overview — 2 Missing DB Indexes — 3 N+1 Query Problem — 4 No Connection Pooling — 5 Missing Cache Layer — 6 No Monitoring — 7 Investigation Timeline — **8 New DB Architecture** — 9 DB Readiness Checklist — 10 10 DB Lessons

"A strong architecture today prevents a midnight outage tomorrow."

DATABASE READINESS CHECKLIST

Use this before every release. Sleep better at night.

Outages are prevented long before they happen.

This checklist is your guardrail.

Your future self will thank you.

HOW TO USE THIS CHECKLIST

1. Review before every release
2. Test under realistic load
3. Fix issues found
4. Monitor continuously

Treat this checklist as a pre-flight checklist for your database.

REAL WORLD IMPACT

- Prevents performance issues
- Avoids midnight pages
- Protects user experience
- Saves engineering time
- Reduces infrastructure cost
- Increases system reliability

Prevention is cheaper than recovery.

1. INDEXING

- ☐ All critical queries are indexed
- ☐ No full table scans in production
- ☐ Indexes are reviewed regularly

Test / Tool

EXPLAIN ANALYZE

2. QUERY PERFORMANCE

- ☐ No N+1 query issues
- ☐ Queries are optimized
- ☐ Slow queries are identified and fixed

Test / Tool

SLOW QUERY LOG
APM / PROFILER

3. CONNECTION MANAGEMENT

- ☐ Connection pooling is enabled
- ☐ Max connections configured properly
- ☐ No connection leaks

Test / Tool

HIKARICP / PGBOUNCER
DB CONNECTION METRICS

4. CACHING

- ☐ Cache layer in place
- ☐ Frequently accessed data is cached
- ☐ Cache invalidation strategy exists

Test / Tool

REDIS METRICS
CACHE HIT RATIO

5. MONITORING & ALERTS

- ☐ Key metrics monitored
- ☐ Alerts configured
- ☐ Dashboards exist and reviewed

Test / Tool

PROMETHEUS / GRAFANA
ALERTMANAGER

6. HIGH AVAILABILITY

- ☐ Replicas are configured
- ☐ Failover tested
- ☐ Backups are automated and verified

Test / Tool

FAILOVER DRILL
BACKUP RESTORE TEST

7. CAPACITY PLANNING

- ☐ Growth is monitored
- ☐ Storage is sufficient
- ☐ Scaling strategy is defined

Test / Tool

CAPACITY DASHBOARD
TREND ANALYSIS

8. SECURITY

- ☐ Access is restricted
- ☐ Secrets are managed securely
- ☐ Encryption is enabled (in-transit & at-rest)

Test / Tool

AUDIT LOGS
SECURITY SCAN

9. DEPLOYMENT SAFETY

- ☐ DB migrations are safe
- ☐ Rollback plan exists
- ☐ Tested in staging with real data

Test / Tool

MIGRATION TESTS
STAGING VALIDATION

10. OBSERVABILITY & INSIGHTS

- ☐ Query insights enabled
- ☐ Performance baselines are known
- ☐ Anomaly detection in place

Test / Tool

DB INSIGHTS / PMM
ANOMALY ALERTS

QUICK HEALTH SCORE

Score your database health (0 - 10)

9 - 10	Excellent
7 - 8	Good
5 - 6	Needs Work
0 - 4	Critical

COMMON RED FLAGS

- ✗ Full table scans
- ✗ High connection count
- ✗ Slow query spikes
- ✗ No monitoring
- ✗ Cache miss rate > 20%
- ✗ Backup failures

BEST PRACTICES

- ✓ Measure everything
- ✓ Automate what you can
- ✓ Test under load
- ✓ Monitor 24/7
- ✓ Review regularly
- ✓ Plan for failure

INTERVIEW NOTE

Q: What would you check before a high traffic release?

A: I would follow a database readiness checklist to ensure performance, reliability, scalability and observability.

KEY LESSON

A checklist today prevents an outage tomorrow.

Discipline > Firefighting.

SERIES PROGRESS

1. Incident Overview
2. Missing DB Indexes
3. N+1 Query Problem
4. No Connection Pooling
5. Missing Cache Layer
6. No Monitoring
7. Investigation Timeline
8. New DB Architecture
9. DB Readiness Checklist
10. 10 DB Lessons

> A checklist is our promise to our future users.

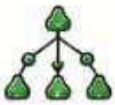
10 DATABASE LESSONS TO NEVER FORGET

From Outage to Insight. From Mistakes to Mastery.

Outages are loud.
Lessons are quiet.
Learn the lessons
so you don't
live the outage
again.
Build it right.
Sleep at night.

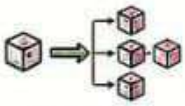
THE OUTAGE TAUGHT US THESE 10 LESSONS

1. INDEX SMARTLY



Good indexes turn hours into milliseconds.

2. AVOID N+1 QUERIES



One request should not mean hundreds of queries.

3. USE CONNECTION POOLING

Connections are expensive. Reuse them.

4. CACHE EVERYTHING

Cache reduces load, improves speed, saves money.

5. MONITOR LIKE YOUR LIFE DEPENDS ON IT

Because one day, it actually will.

6. MEASURE EVERYTHING

You can't improve what you don't measure.

7. DESIGN FOR SCALE

Today's queries must handle tomorrow's scale.

8. TEST UNDER LOAD

If it breaks under load in testing, it will break in prod.

9. AUTOMATE RECOVERY

Automate backups, failovers, and healing.

10. OBSERVABILITY IS NOT OPTIONAL

Metrics, logs, traces – your system's early warning system.

BEFORE vs AFTER MINDSET

BEFORE (What broke us)	AFTER (What will protect us)
✗ No indexes	✓ Proper indexing
✗ N+1 queries	✓ Optimized queries
✗ No connection pooling	✓ Connection pooling
✗ No cache	✓ Caching layer
✗ No monitoring	✓ Full monitoring
✗ Single DB instance	✓ Primary + replicas
✗ No visibility	✓ End-to-end visibility
✗ Outages & slowdowns	✓ High availability

OUR JOURNEY

1. Incident Overview
2. Missing DB Indexes
3. N+1 Query Problem
4. No Connection Pooling
5. Missing Cache Layer
6. No DB Monitoring
7. Investigation Timeline
8. New DB Architecture
9. Readiness Checklist
10. 10 Lessons Learned

From Outage to Excellence

OUTAGE PREVENTION EQUATION

BUSINESS IMPACT WE PROTECTED

- Users happy
- Revenue protected
- Engineers productive
- Systems reliable

→ Trust Earned

QUICK REFERENCE CHEAT SHEET

QUERY PERFORMANCE

- Index frequently
- Avoid N+1 queries
- Use EXPLAIN ANALYZE
- Optimize slow queries

CONNECTION MANAGEMENT

- Use connection pooling
- Set max connections
- Monitor connection usage
- Prevent connection leaks

CACHING

- Cache read-heavy data
- Use Redis
- Set proper TTL
- Invalidate smartly

MONITORING

- Monitor key metrics
- Set alerts early
- Track query performance
- Build dashboards

RELIABILITY

- Replicate data
- Automate backups
- Test failover
- Plan for disasters

TOOLS WE LOVE

- PostgreSQL
- Redis
- Prometheus + Grafana
- pg_stat_statements
- P6Spy / Datasource Proxy

REMEMBER

The best database is not the one with the most features. It's the one that is designed, monitored, and cared for.

INTERVIEW NOTE

Q: What is the most important database lesson from this outage?

A: Small mistakes (no index, N+1, no cache, no monitoring) combine to create major outages. Prevention is always cheaper.

KEY LESSON

Databases don't fail suddenly.
They slow down silently.
Monitor. Optimize. Repeat.

FINAL WORD

"We don't just fix outages.
We build systems that make outages unlikely."

SERIES COMPLETE

1. Incident Overview
2. Missing DB Indexes
3. N+1 Query Problem
4. No Connection Pooling
5. Missing Cache Layer
6. No Monitoring
7. Investigation Timeline
8. New DB Architecture
9. DB Readiness Checklist
10. 10 DB Lessons

"Great databases are not born. They are designed, monitored and evolved.
Don't wait for an outage to learn these lessons."